

Curso <i>Engenharia de Software</i>	Campus <i>Rio Verde</i>
Disciplina <i>Programação Orientada a Objetos</i>	
Nome do(a) acadêmico(a) GABARITO OFICIAL	Valor 7,0
Professor <i>Sergio Souza Novak</i>	Turma <i>B</i>
Data da Avaliação <i>13/05/2026</i>	Código <i>ESW448</i>

QUESTÕES TEÓRICAS

1. Resposta: B

- Afirmativa 1: Falsa. Classes abstratas podem conter atributos normalmente.
- Afirmativa 2: Verdadeira. Podem conter tanto métodos abstratos quanto concretos.
- Afirmativa 3: Falsa. Construtores não podem ser abstratos.
- Afirmativa 4: Falsa. Classes abstratas não podem ser instanciadas diretamente.

2. Resposta:

RESPOSTA:

O modificador `private` restringe o acesso ao atributo `nome` apenas à própria classe `Animal`, impedindo o acesso direto pelas subclasses. O modificador `protected` permite que o atributo `idade` seja acessado por `Animal`, suas subclasses (`Cachorro` e `Gato`) e outras classes do mesmo pacote. O modificador `public` torna o método `emitirSom()` acessível de qualquer lugar. No método `exibirDados()`, o atributo `nome` **não** pode ser acessado diretamente devido ao encapsulamento (`private`), enquanto o atributo `idade` **pode** ser acessado diretamente por conta da herança e do modificador `protected`.

3. Resposta:

- (V) Para que ocorra a sobrescrita, o método na subclasse deve possuir o mesmo nome e a mesma assinatura (parâmetros) do método na superclasse.
- (F) Um método definido como `final` na superclasse pode ser livremente sobrescrito... (*Métodos final não podem ser sobrescritos*).
- (V) O uso da anotação `@Override` é opcional, mas ajuda o compilador...
- (V) A sobrescrita permite que uma subclasse forneça uma implementação específica...
- (F) Se um método for declarado como `static` na superclasse, ele pode ser sobrescrito... (*Métodos estáticos são ocultados, não sobrescritos*).

4. **Resposta:**

RESPOSTA:

- a) O compilador baseia-se no tipo da referência (`Animal`) para verificar a validade das chamadas. Como `Animal` não possui o método `enterrarOsso()`, a chamada direta é negada por segurança de tipo.
- b) Ocorreria uma exceção do tipo `ClassCastException` em tempo de execução, pois um objeto do tipo `Gato` não pode ser convertido para `Cachorro`, já que não há relação de herança entre eles (são irmãos na hierarquia).

5. **Resposta: A**

- I: Correta. `final` em classes impede herança.
- II: Incorreta. `final` em métodos impede sobrescrita.
- III: Correta. `final` em atributos cria constantes (após inicialização).
- IV: Correta. Sem herança, não há sobrescrita.

QUESTÕES PRÁTICAS

1. Implementação de ItemVestuario:

RESPOSTA:

```
public abstract class ItemVestuario {  
    protected String marca;  
    protected double preco;  
    public static final String MOEDA = "R$";  
    public static int totalItens = 0;  
  
    public abstract void exibirDetalhes();  
}
```

2. Implementação de Camiseta:

RESPOSTA:

```
public class Camiseta extends ItemVestuario {
    private String tamanho;
    public static final String TIPO = "Camiseta";

    public Camiseta(String marca, double preco, String tamanho) {
        this.marca = marca;
        this.preco = preco;
        this.tamanho = tamanho;
        totalItens++;
    }

    @Override
    public void exibirDetalhes() {
        System.out.println("Tipo: " + TIPO + " Marca: " + marca +
            " Preço: " + MOEDA + preco + " Tam: " + tamanho);
    }
}
```

3. Implementação de Blusa:

RESPOSTA:

```
public class Blusa extends ItemVestuario {
    private String tecido;
    public static final String TIPO = "Blusa";

    public Blusa(String marca, double preco, String tecido) {
        this.marca = marca;
        this.preco = preco;
        this.tecido = tecido;
        totalItens++;
    }

    @Override
    public void exibirDetalhes() {
        System.out.println("Tipo:  "+ TIPO +  Marca:  "+ marca +
            Preço:  "+ MOEDA + preco +  Tecido:  "+ tecido);
    }
}
```

4. Downcasting e Explicação:**RESPOSTA:**

```
((Camiseta) item).dobrarCamiseta();
```

Os parênteses externos são necessários devido à precedência de operadores. O operador de ponto (.) tem precedência maior que o operador de *cast*. Sem os parênteses, o compilador tentaria primeiro executar o método no objeto do tipo *ItemVestuario* antes de realizar a conversão.

5. SOLID e Classes Abstratas:

RESPOSTA:

As classes abstratas são fundamentais para o Princípio do Aberto/Fechado (OCP), permitindo que o comportamento de um sistema seja estendido sem modificar o código existente, e para o Princípio da Inversão de Dependência (DIP), pois garantem que módulos de alto nível dependam de abstrações e não de implementações concretas, promovendo baixo acoplamento.